Trace Verbosity Degrades SFT Verdict Prediction: A Case Study in Information Density Effects

Anonymous ACL submission

Abstract

A natural assumption when augmenting code models with execution traces is that richer runtime information yields better predictions. We challenge this for verdict prediction (classifying whether a test passes or fails) and find the opposite: complete traces degrade accuracy to near-baseline levels, while compact exception events alone reach $\geq$99.33% across all models tested. Through the first systematic component ablation of trace representations across multiple code models and architecture families, we identify information density as the key factor: exception events encode the fault signal in roughly 150 tokens, whereas full traces dilute it across thousands of routine steps, trapping models in unstable oscillation. Controlled experiments support this finding: gradient masking isolates backward-pass dilution as a minor factor, a tail-truncation sweep reveals a sharp phase transition at the diagnostic-event boundary, and format-marker removal confirms the signal is semantic. These results decompose trace verbosity into a two-tier diagnostic hierarchy and establish that targeted signal extraction strictly dominates richer input for structurally constrained tasks such as verdict prediction[1].

1 Introduction

Execution traces record a program's complete runtime behavior: every function call, variable assignment, and control flow decision. Recent work has incorporated them into code language models for tasks from bug localization to program repair (Chen et al., 2025; Ding et al., 2024; Wang et al., 2025a; Ni et al., 2024), under the working assumption that richer runtime information yields better predictions. *We find the opposite.* For verdict prediction (classifying whether a test passes or fails), feeding a model the complete execution trace produces accuracies of 79–82%, barely above a source-code-only baseline (72–76%), while a single compact component of the same trace reaches $\geq$99.33% across all models tested. Verbose traces do not just fail to help; they actively impair learning.

Verdict prediction offers a clean testbed for isolating this effect. As binary classification, it separates signal extraction from generation noise; as fault detection, the model need only detect failure, not simulate execution. If the fault signal concentrates in a small trace subset, information density should matter more than quantity. Prior work found no single trace representation consistently outperforms others (Wang et al., 2025a), but focused on generation tasks where this density hypothesis could not be cleanly tested.

We conduct the first systematic component ablation of trace representations, decomposing VOTA (exception events + assertion comparisons + loop compression) into individual components across four models spanning 1.5B to 13B parameters and three architecture families (Team, 2024; Rozière et al., 2023; Guo et al., 2024), on 591 problems from HumanEval (Chen et al., 2021) and MBPP-sanitized (Austin et al., 2021). The results are unambiguous: exception events (a median of 150 tokens encoding error type, stack frame, and message) reach $\geq$99.33% accuracy across all models and seeds, converging within 50 training steps. Full traces plateau near 82% and gradient-masked variants reach only $\sim$86%, both without stable convergence. This confirms that information density, not model capacity, is the binding constraint (Figure 7).

This finding is robust to several potential confounds. Removing format markers preserves accuracy within 2pp, and on 8 boundary samples where exceptions appear in passing code, the fine-tuned model correctly classifies 6/8 vs. 0/8 for a regex baseline, which confirms the signal is semantic (§4.3). Gradient masking bounds the backward-pass contribution to $\sim$4pp (§4.4), a tail-truncation budget sweep across four models reveals a phase

[1]Code and data are available at https://anonymous.4open.science/r/trace-verdict-study-291F/.

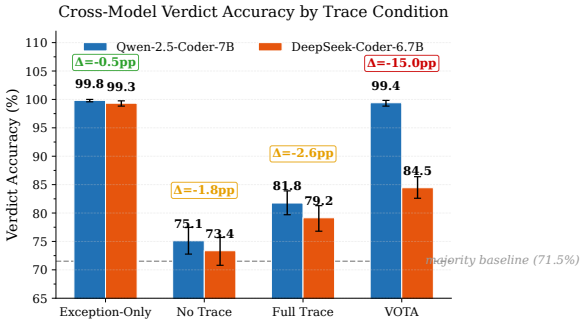


Figure 1: Cross-model verdict accuracy by trace condition (Qwen-2.5-Coder-7B vs. DeepSeek-Coder-6.7B, N=1,190). *exception_only* shows near-identical accuracy across models (Δ=0.5pp), while *VOTA* exhibits a 15pp cross-model gap due to a tokenizer-induced training failure (Appendix F).

transition at ∼500 tokens (§4.5), and cross-dataset transfer to DebugBench (N=426) preserves the component ranking (§4.5).

Our contributions:

1. **Information density as the binding constraint on SFT learning**: training dynamics, gradient masking (∼4pp backward-pass contribution), and cross-model convergence confirm that compact trace inputs learn faster and more stably than verbose ones, independent of model scale (§4.2, §4.4).

2. **Exception events as the minimal sufficient signal**: ≥99.33% across all four models with zero cross-seed variance on Qwen and sub-1pp cross-model range (§4.1).

3. **A two-tier diagnostic hierarchy with quantitative decomposition**: Tier 1 signals (exceptions, assertion comparisons) are individually sufficient for near-perfect accuracy and generalize across architectures within a 0.50–1.11pp cross-model range, while Tier 2 signals (loop compression) are model-dependent with ∼9pp range; a sharp phase transition at ∼500 tail tokens spatially localizes the diagnostic boundary (§4.2, §4.5).

2 Related Work

2.1 Execution Traces for Code LLMs

Recent work has scaled trace-augmented training: TracePile (Chen et al., 2025) pre-trained on 2.6M execution-annotated samples, while Liu et al. (2023) pre-trained models to predict execution outputs from traces. Wang et al. (2025a) found trace utility to be task- and format-dependent but did not decompose individual trace components; relative to their work, we contribute a two-tier signal hierarchy, a phase transition in trace-length budgets, calibration analysis, and cross-architecture validation. NExT (Ni et al., 2024) and ET-CoT (Eguchi and Akiba, 2025) used trace chain-of-thought for execution reasoning, and Armengol-Estape et al. (2025) found compact traces comparable to full traces, consistent with our density finding. CRUX-Eval (Gu et al., 2024), extended to 19 languages by CRUXEval-X (Xu et al., 2025), established execution reasoning benchmarks, though neither decomposes which trace components contribute to *classification* accuracy. Shi et al. (2023) showed LLMs are easily distracted by irrelevant context, providing a complementary explanation for why verbose traces impair SFT learning. In the prompt compression literature, LLMLingua (Jiang et al., 2023) and Selective Context (Li et al., 2023) showed that pruning low-information tokens preserves downstream accuracy; TraceCoder (Huang et al., 2026) and InspectCoder (Wang et al., 2025c) apply selective trace extraction for automated debugging; and Haque et al. (2025) found raw traces provide limited repair benefit. Our density decomposition complements these methods by quantifying *why* compression helps (gradient dilution explains ∼24% of the gap; the remainder reflects inference-time interference).

2.2 Verdict Prediction and Fault Detection

Verdict prediction is an instance of the long-standing test oracle problem (Barr et al., 2015). TRACED (Ding et al., 2024) showed execution-aware pretraining improves vulnerability detection; DeepDFA (Steenhoek et al., 2024) and learned program slicing (Yadavally et al., 2024) combined classical program analysis with deep learning for defect detection. In patch correctness prediction, representation learning of code changes has progressed from CC2Vec (Hoang et al., 2020) to LLM4PatchCorrect (Zhou et al., 2024) (84.4%) and APPT (Zhang et al., 2024) (79.7%); Tian et al. (2020) evaluated change representations for predicting repair correctness. FuseFL (Widyasari et al., 2024) and Foster et al. (2025) demonstrated that structured fault signals (stack traces, mutation outcomes) improve localization; the equivalent-mutant problem (Papadakis et al., 2019) poses an analogous classification challenge. Spectrum-based fault localization (SBFL) ranks program elements by

Table 1: Cross-model verdict accuracy (%, N=1,190) across four models and three architecture families. Bold: best per model. ‡2-seed mean (seed 42 training failure; Appendix F). Multi-seed variance in Appendix E.

Condition	Qwen-7B	Qwen-1.5B	CL-13B	DS-6.7B
exception_only	**99.83**	**99.83**	**99.89**	**99.39**
ae_only	99.64	98.83	99.08	98.53
VOTA	99.05	99.54	98.74	99.25‡
loop_only	86.05	79.54	77.40	80.31
no_trace	75.13	72.35	76.47	73.36
Majority baseline		71.51		

suspiciousness from test spectra (Abreu et al., 2007; Jones et al., 2002; Wong et al., 2014); recent LLM-based approaches such as AutoFL (Kang et al., 2024) and AgentFL (Qin et al., 2024) share our insight that anomalous execution events carry the strongest diagnostic signal. In agent-based repair, RepairAgent (Bouzenia et al., 2025), Agentless (Xia et al., 2024), SWE-agent (Yang et al., 2024b), and OpenHands (Wang et al., 2025b) all incorporate execution feedback in repair workflows; in the SWE-bench ecosystem (Jimenez et al., 2024), these agents rely on test verdict signals to guide patch selection, yet which trace components carry that signal most efficiently remains unstudied. LDB (Zhong et al., 2024) applies segment-level trace verification for debugging, and SelfDebug (Chen et al., 2024) and Reflexion (Shinn et al., 2023) iteratively refine code using execution outcomes. None of these works isolate individual trace components or compare against rule-based baselines.

2.3 Calibration and Training Dynamics

Neural network calibration (Guo et al., 2017), extended to LLMs (Kadavath et al., 2022; Geng et al., 2024), studies confidence–correctness alignment; Spiess et al. (2025) specifically addressed calibration for code models. Calibration analysis of trace-based code models remains largely unexplored. Just-in-time defect prediction (Kamei et al., 2013; Hoang et al., 2019; Pornprasit and Tantithamthavorn, 2021) shares our fault detection motivation but operates on code change features rather than execution traces; our work complements this line by showing that execution-time fault signals can be classified with near-perfect accuracy when properly extracted. Curriculum learning (Bengio et al., 2009), dataset cartography via training dynamics (Swayamdipta et al., 2020), and data selection methods (Sachdeva et al., 2024; Xie et al., 2023) demonstrate that data quality and ordering improve convergence, consistent with our finding that compact exceptions enable convergence within 50 steps while verbose traces trap the model in oscillation across all 444 training steps. Yang et al. (2024a) further showed that fine-tuned LLMs generalize better on concise inputs, providing a complementary perspective on why information density governs SFT effectiveness.

3 Method

3.1 Task Definition

We study *verdict prediction*: given a Python function, a test input, and an optional execution trace, predict whether the test passes or fails. The model produces a single token (*pass* or *fail*), making this a binary classification task. Unlike code generation (Chen et al., 2025) or output prediction (Gu et al., 2024), verdict prediction isolates the model's ability to detect faults from its ability to simulate execution. The discrete binary outcome makes the effect of each trace component directly measurable, providing cleaner ablation contrasts than generation tasks where output quality varies continuously.

3.2 Dataset

We sampled 10 candidate solutions per problem from Qwen-2.5-Coder-7B-Instruct (temperature 0.8, top_p 0.95) for 591 problems from HumanEval (Chen et al., 2021) (164) and MBPP-sanitized (Austin et al., 2021) (427), executed each against canonical tests with *sys.settrace* to obtain line-level traces and pass/fail verdicts, yielding 5,910 pairs (67% pass, 33% fail). The dataset is split 80/20 by problem ID into 4,720 training and 1,190 test samples (majority baseline: 71.51%). The *exception_only* format collapses passing solutions into identical rows (39.45% train, 48.99% test duplicates); these duplicates arise from multiple candidate solutions per problem, with the train–test split enforced at the problem level (472 train / 119 test problems, zero crossover; Appendix O).

3.3 Trace Representations

We define seven conditions that vary the trace information available:

- *no_trace*: source code and test input only.
- *full_trace*: complete line-by-line trace with variable states.
- *VOTA*: exception events + assertion-point comparisons + loop compression (Appendix C).
- *exception_only*: exception and error events only.

Table 2: Component ablation on Qwen-2.5-7B (N=1,190). Tier 1 signals achieve near-perfect accuracy; combining them (VOTA) slightly dilutes performance. Entries marked †/§/‡ are 3-seed mean±std; per-seed breakdowns in Appendix E. Seed σ is lowest for *exception_only* (σ=0.00) and highest for *loop_only* (σ=0.52); *ae_only* (σ=0.49) is an outlier, suggesting seed sensitivity tracks input length rather than signal density alone.

Condition	Components	Acc (%)	95% CI	Tier
ae_only	assertion-point	99.64†	[99.26, 99.87]	1
exception_only	exception events	**99.83**	[99.58, 100.00]	1
VOTA	exc+ae+loop	99.05§	[98.49, 99.50]	—
VOTA_no_ae	exc+loop	99.41	[98.90, 99.80]	—
loop_only	loop compression	86.05‡	[83.97, 87.96]	2
full_trace	complete trace	81.76	[79.70, 83.90]	—
no_trace	none	75.13	[72.77, 77.48]	—
Regex (best)	exc. type regex	93.61	—	—
3-shot (Qwen-7B)	in-context learning	90.84	—	—
Zero-shot (4 models)	prompting	51.0–93.5	—	—

- *ae_only*: assertion-point actual/expected pairs only.
- *loop_only*: loop compression summaries only.
- *VOTA_no_ae*: VOTA without assertion-point information.

3.4 Models and Training

We evaluate four models spanning three architecture families: **Qwen-2.5-Coder-7B/1.5B-Instruct** (Team, 2024), **CodeLlama-13B-Instruct** (Rozière et al., 2023), and **DeepSeek-Coder-6.7B-Instruct** (Guo et al., 2024), covering a 1.5B–13B parameter range (8.7×). All conditions use LoRA (Hu et al., 2022) (rank=16, α=32, dropout=0.05) on all linear layers, 3 epochs, effective batch size 32, LR 2×10^{-4} with linear warmup/decay, max sequence length 2,048 tokens. Sequences exceeding this limit are truncated while preserving the verdict label. The primary seed is 42; seeds 123 and 456 test *exception_only* robustness (Appendix B).

3.5 Loss Ablation

To disentangle training signal from input signal quality, we introduce *full_trace_label_only*: identical full-trace input but loss computed only on the verdict token (trace positions masked). Improvement over standard *full_trace* is attributable to backward-pass signal dilution; the remaining gap to VOTA reflects inference-time input complexity.

4 Experiments

We evaluate two questions: (1) Do exception signals generalize across model architectures and scales? (2) What is the relative contribution of each trace component? All experiments use N=1,190 test samples with Wilson 95% CIs.[2] All cross-tier differences are significant at $p<.0001$ (paired bootstrap, B=10,000) unless noted. Under problem-level clustered bootstrap (effective $N\approx119$), cross-tier gaps remain significant ($p<.001$); within-tier comparisons show partially overlapping CIs.

4.1 Cross-Model Comparison

Table 1 compares four models spanning three architecture families across a 1.5B–13B parameter range.

Exception dominance across families. Under *exception_only*, all four models converge to ≥99.33% across all seeds, with a cross-model range of just 0.50pp, the tightest of any condition. CodeLlama-13B reaches 100% on one seed; the two persistent errors on Qwen (both *GeneratorExit*; Appendix G) are convergence artifacts rather than hard data limitations.

Scale invariance. Exception accuracy is independent of model scale: the smallest model (1.5B) matches the largest (13B) on the same representation (Table 1). In contrast, *no_trace* scales monotonically with capacity and exhibits the highest seed variance of any condition. Exception events eliminate both the scale gap and the instability.

VOTA robustness. Excluding one training failure caused by tokenizer-induced truncation (Appendix F), VOTA achieves ≥98.57% across all models within a 0.80pp range. The failure is itself informative: 91.7% of that seed's training samples were truncated, while compact *exception_only* inputs (median 150 tokens) sidestep truncation entirely.

4.2 Component Ablation

Table 2 decomposes VOTA on Qwen-2.5-7B, revealing a two-tier hierarchy. Tier 1 signals are individually sufficient: *exception_only* alone exceeds full VOTA, implying that combining additional components introduces noise rather than value. Removing assertion points has no effect (*VOTA_no_ae*), which reaffirms exception dominance. The Tier 1/2 boundary is sharp (Figure 2): the weakest Tier 1 condition exceeds the strongest Tier 2 by over 12pp with non-overlapping confidence intervals. Tier 2 (*loop_only*) benefits are

[2] Problem-level split (472/119 problems, zero crossover). Deduplication sensitivity analysis confirms 0.00pp delta across all 19 conditions (Appendix O).

Table 3: Information density decomposition on Qwen-7B. Gradient masking (*label_only*) bounds backward-pass dilution; the remaining gap reflects inference-time input factors. 3-seed means.

Condition	Acc (%)	Δ vs. full_trace	Gap explained
full_trace	81.76	—	—
label_only	85.94±1.12	+4.18pp	24%
VOTA	99.05±0.38	+17.29pp	100%

model-dependent: Qwen-7B gains +10.92pp over *no_trace* while CodeLlama gains only +0.93pp (n.s.), suggesting loop utilization reflects architectural capacity to exploit structured iteration patterns rather than a universal signal. Within Tier 2, the larger Qwen-7B outgains Qwen-1.5B (+10.92 vs. +7.19pp, $p<.0001$), the only condition where scale provides a meaningful advantage.

Baselines. The strongest regex heuristic (exception class name matching) trails SFT *exception_only* by 6.22pp ($p<.001$); the gap concentrates on boundary cases where SFT distinguishes caught from uncaught exceptions, correctly resolving 6/8 ambiguous samples vs. 0/8 for regex (Appendix P). Zero-shot prompting spans a 42pp range across models; SFT compresses this to 0.50pp with 4,720 training samples (Figure 6; Appendix Q).

4.3 Format Marker Ablation

The *exception_only* format includes a "[No exception data in trace]" marker for pass-class samples. Retraining with all format markers removed (*no-marker*) yields a modest drop (∼2pp on both Qwen-7B and Qwen-1.5B), accounting for only ∼9% of the total exception lift over *no_trace*. Exception events thus provide two signal layers: structural markers (∼2pp, regex-learnable) and semantic content (∼22pp, requiring SFT to distinguish caught from uncaught exceptions). Both layers show no degradation at smaller scale.

4.4 Information Density Decomposition

The gap between *full_trace* and VOTA decomposes via *full_trace_label_only*, which retains the full trace input but masks all trace tokens from the loss (Table 3).

Gradient masking places an upper bound on backward-pass dilution: the +4.18pp gain bounds the backward-pass contribution to ∼24% of the full gap. The remaining majority of the gap persists even with identical loss computation, suggesting that interference from thousands of irrelevant trace

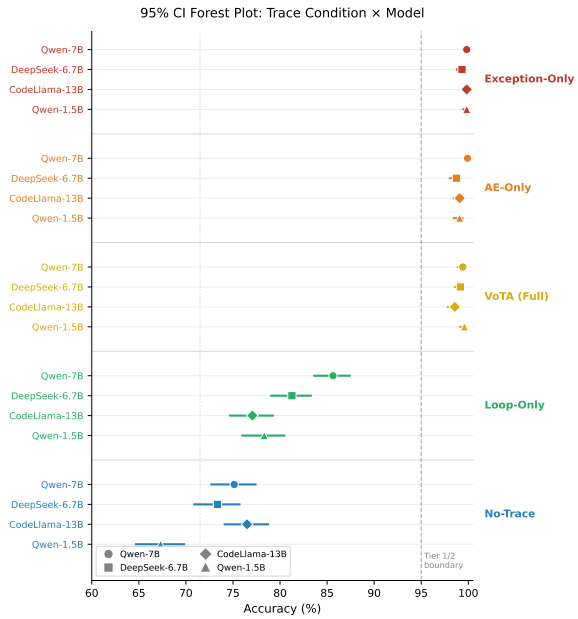


Figure 2: Cross-model accuracy with 95% Wilson CIs across all conditions and four models. The shaded region highlights the Tier 1/2 boundary gap (>12pp). Within Tier 1, all models converge to ≥99.08%; within Tier 2, performance is model-dependent.

lines at inference, rather than training signal dilution, is the dominant factor. Cross-model evidence corroborates this: the tail-truncation phase transition at ∼500 tokens (§4.5) coincides with exception traceback length, and cross-scale validation on Qwen-1.5B yields a consistent pattern (gradient masking explains a similar minority share). CodeLlama and DeepSeek exhibit label-only collapse, limiting the decomposition to models whose tokenizers preserve sufficient trace structure. Error analysis confirms all *full_trace* errors occur where truncation removes diagnostic markers (Figure 3). The two factors may interact, and the decomposition is approximate (Qwen family, 2 scales, 3 seeds).

4.5 Additional Analyses

Tail-truncation budget curve. A cross-model budget sweep reveals a sharp phase transition between 384 and 512 tail tokens (Figure 3). Below this threshold, all models remain near baseline regardless of how many tokens are retained. Above it, accuracy jumps by 13–22pp in a single step, converging into a narrow band by tail-1024. The transition zone coincides with the typical exception traceback length (∼400–500 tokens), providing spatial evidence that diagnostic events in the trace tail drive the accuracy gains.

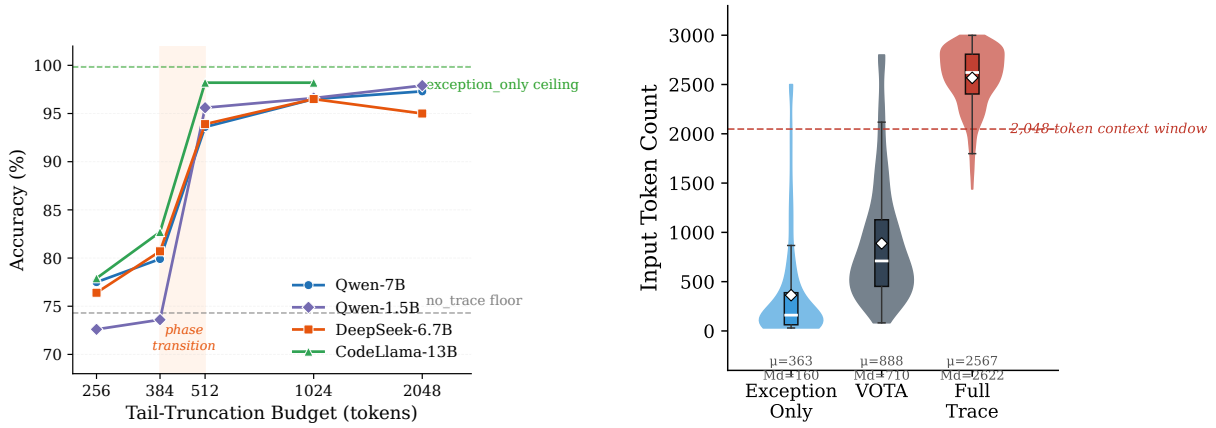


Figure 3: (**Left**) Cross-model tail-truncation budget curve (N=1,190, four models). All models remain near baseline at tail-256/384, then jump sharply at 384→512 as the exception traceback is captured (shaded). Dashed: *exception_only* range; dotted: *no_trace* range. (**Right**) Input token length distribution across trace conditions (training set, N=4,720). Dashed line: 2,048-token context window. *exception_only* inputs (median 150 tokens) fit entirely within the window, while *full_trace* inputs frequently exceed it, triggering truncation.

Tail-truncation dramatically outperforms head-truncation: retaining the last k tokens exceeds retaining the first k by 15–23pp across models. Under head-truncation, accuracy is non-monotonic and confined to a narrow band, because mid-length prefixes retain enough irrelevant content to mislead the model while discarding diagnostic events near the trace terminus (Appendix I).

Calibration. Calibration mirrors the tier hierarchy: *exception_only* achieves an Expected Calibration Error (ECE) of 0.003 vs. 0.171 for *no_trace*, a 63× gap (Figure 4). This advantage generalizes across all four models (ECE $<$ 0.007; Appendix J).

Error orthogonality and exception types. *exception_only*'s residual errors have zero overlap with all other conditions' error sets (Jaccard = 0.000; Figure 5), which confirms it operates on an independent signal dimension. All standard exception types achieve 100% accuracy; the only failures arise from *GeneratorExit*, a benign *BaseException* subclass (Appendix H).

Cross-dataset generalization. DebugBench (Tian et al., 2024) (N=426) preserves the component ranking (*exception_only* 100%, VOTA 99.77%, *no_trace* 96.71%), though narrow margins reflect a ceiling effect from structurally distinctive mutations (Appendix L).

5 Discussion

The central finding is that information density governs learning efficiency for trace-based verdict prediction. Exception events concentrate the fault signal in a compact format (median 150 tokens), while non-exception components record *how* the program ran, not *whether* it failed. This explains three patterns: (1) adding information beyond exceptions provides no benefit (indeed, VOTA underperforms *exception_only*, Table 2); (2) the tail-truncation phase transition at ∼500 tokens coincides with the diagnostic-event boundary (Figure 3); and (3) the SFT advantage over regex concentrates on caught-exception boundary cases (§4.3). The narrow cross-model gap (Table 1) supports density, not capacity, as the binding constraint.

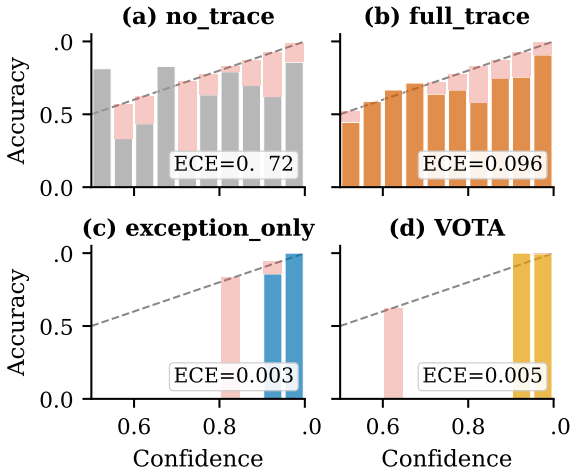


Figure 4: Reliability diagrams for four trace conditions on Qwen-2.5-7B (N=1,190). *exception_only* (ECE=0.003) clusters near the diagonal; *no_trace* (ECE=0.171) shows systematic overconfidence.

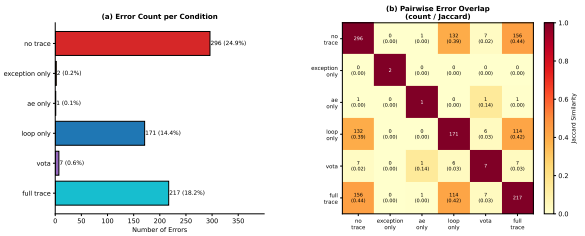


Figure 5: Pairwise Jaccard similarity between error sets (Qwen-7B, N=1,190). *exception_only* has zero overlap with all other conditions and operates on an independent signal dimension.

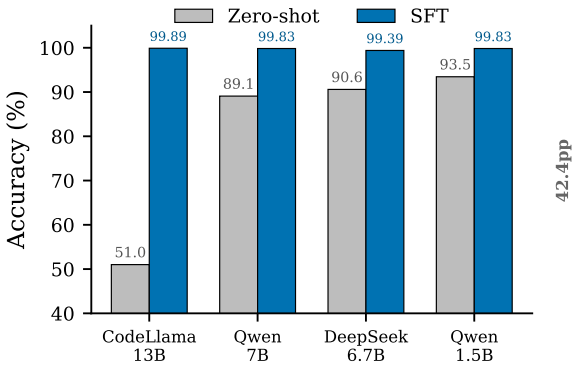


Figure 6: Zero-shot vs. SFT *exception_only* accuracy across four models (N=1,190). SFT compresses the cross-architecture variance from 42.44pp to 0.50pp.

SFT vs. regex: semantic understanding. Format-marker removal drops accuracy by only ∼2pp, leaving a clear gap over the best regex. On boundary samples with caught exceptions, SFT correctly classifies 6/8 vs. 0/8 for regex (p=.041). This confirms the signal is semantic rather than syntactic. This distinction matters for deployment: regex baselines can detect most exceptions through surface-level pattern matching, but fail precisely on the ambiguous cases (caught exceptions within *try/except* blocks) where correct classification requires understanding the exception's role in program control flow. SFT learns this distinction from fewer than 5,000 training examples, suggesting that the relevant semantic features are readily extractable from pretrained code representations when the input is sufficiently compact.

Near-perfect floor. The two residual errors (both *GeneratorExit*; Appendix K) are shared across configurations and models. One seed achieves 100%, which establishes 99.83% as a convergence *floor*, not a ceiling. The error overlap analysis (Figure 5) reveals that these errors are orthogonal to all other conditions' error sets (Jaccard = 0.000); *exception_only* therefore operates on an independent signal dimension. The practical implication is that the two residual errors cannot be resolved by adding more trace information; they require disambiguating benign *BaseException* subclasses from genuine faults, a representation-level fix rather than a data quantity issue.

Tier hierarchy and design principles. The two-tier structure (Figure 2) suggests a hierarchical extraction strategy: extract exception events first, resort to Tier 2 signals only when no exception is raised. Within Tier 1, both *exception_only* and *ae_only* achieve near-perfect accuracy independently, yet combining them in VOTA slightly *degrades* performance, a counter-intuitive finding that underscores how additional signal can introduce noise when the primary signal is already sufficient. Loop benefit is model-dependent (Qwen-7B gains +10.92pp while CodeLlama gains only +0.93pp). This reflects an architecture-level capacity to exploit structured iteration patterns rather than a universal signal. This density–efficiency link should transfer to any trace-based task where compact diagnostic events exist within verbose runtime logs, such as crash triage in continuous integration, where stack traces carry far more signal than full execution histories.

Cross-model robustness and scale independence. The cross-model analysis reveals that *exception_only*'s accuracy range is <1pp across four models (1.5B–13B) vs. ∼9pp for *loop_only*, demonstrating that compact inputs eliminate the architecture-dependent variance that plagues verbose representations. This scale independence is striking: the smallest model (Qwen-1.5B at 1.5B parameters) matches the largest (CodeLlama-13B at 13B parameters) when given the same compact input, whereas *no_trace* accuracy scales monotonically with model capacity (Table 1). The zero-shot comparison reinforces this: SFT compresses a 42.44pp cross-architecture range to just 0.50pp with only 4,720 training samples (Figure 6), making exception-based verdict prediction accessible to smaller models without sacrificing accuracy. Label-only loss on high-truncation tokenizers causes model collapse, further confirming density as the binding constraint: when the tokenizer is inefficient, even masking trace tokens from the loss cannot compensate for the inference-time interference from thousands of irrelevant tokens.

Exception events capture test verdicts, not just crashes. A natural concern is that exception-based classification merely detects program crashes. In fact, 55.5% of failures in our dataset are silent logic errors where the program terminates normally but produces incorrect output. *exception_only* nonetheless achieves 100% fail recall because test assertions generate *AssertionError* events that explicitly mark the discrepancy between actual and expected behavior. This means the "exception" signal subsumes both crash-type and assertion-type failures, making it a complete verdict signal rather than a crash detector.

Alternative explanations. Three alternatives merit discussion. *Length asymmetry*: padding all inputs to the maximum sequence length leaves accuracy unchanged, ruling out a confound from batch-level length variation. *Format-marker familiarity*: removing markers accounts for only ∼9% of the exception lift, though familiarity with exception formats in pretraining data cannot be fully controlled for. *Positional concentration*: diagnostic information concentrates in the trace tail (§4.5); the tail-truncation budget curve localizes the critical threshold to ∼400–500 tokens, consistent with typical exception traceback length, but fully disentangling positional proximity from semantic density requires future work with controlled position-shuffled experiments.

Connections to prompt compression. Our findings align with the prompt compression literature: LLMLingua (Jiang et al., 2023) and Selective Context (Li et al., 2023) showed that pruning low-information tokens preserves downstream accuracy. The trace domain provides a natural instance of this phenomenon: exception events are effectively a domain-specific compression that retains the diagnostic tokens and discards the execution log. The information density decomposition (§4.4) quantifies this: removing all non-diagnostic tokens accounts for the majority of the performance gap, while backward-pass gradient dilution explains at most 24%.

Learning dynamics. *exception_only* converges within 50 steps and saturates by step 150, while *full_trace* oscillates across all 444 training steps without reaching stable convergence (Figure 7). Seed variance negatively correlates with input density (Table 2): *exception_only* shows zero cross-seed variance (σ=0.00) while *loop_only* exhibits

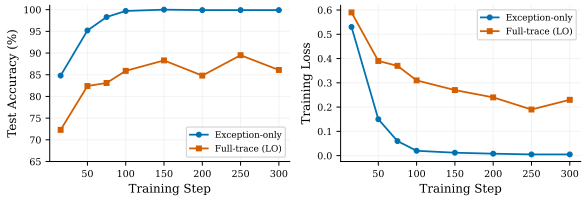


Figure 7: Training dynamics comparison. *exception_only* (blue) converges within 50 steps and saturates by step 150. *full_trace* with label-only loss (red, dashed) oscillates without stable convergence across all 444 training steps.

the highest (σ=0.52). This pattern suggests that compact inputs not only improve final accuracy but also stabilize the optimization landscape, reducing sensitivity to random initialization. From a practical standpoint, this means exception-based models can be trained with a single seed and minimal hyperparameter tuning, whereas verbose-trace models require multiple seeds and careful early-stopping selection.

6 Conclusion

We presented the first systematic component ablation of trace representations for verdict prediction, covering four code models across three architecture families. The finding is clear: information density, not trace completeness, determines learning efficiency under SFT. Compact exception events converge to ≥99.33% across all models, while verbose full traces oscillate without stable convergence. These results establish a two-tier diagnostic hierarchy: Tier 1 signals (exceptions, assertion comparisons) generalize across architectures within a 0.50–1.11pp range, while Tier 2 signals (loop patterns) are model-dependent. For structurally constrained tasks such as verdict prediction, targeted signal extraction should replace wholesale trace inclusion.

Limitations

Our study has three main limitations. First, all experiments use Python execution traces from code-specialized models ranging from 1.5B to 13B parameters; whether the observed density–efficiency relationship holds for other programming languages, human-written code, or larger models (30B+) remains untested. Second, our findings may not generalize beyond the current setting: Python's structured exception hierarchy makes exception events particularly informative, and lan-

guages with less structured error reporting may exhibit different signal distributions. Third, the gradient masking experiment isolates backward-pass signal dilution but does not quantify forward-pass attention effects; disentangling these two factors requires further investigation.

References

Rui Abreu, Peter Zoeteweij, and Arjan J.C. van Gemund. 2007. On the accuracy of spectrum-based fault localization. *Testing: Academic and Industrial Conference Practice and Research Techniques (MUTATION)*.

Jordi Armengol-Estape, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F.P. O'Boyle, Sida Wang, Gabriel Synnaeve, and Hugh Leather. 2025. What i cannot execute, i do not understand: Training and evaluating LLMs on program execution traces. *arXiv preprint arXiv:2503.05703*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Nuo Chen, Zehua Li, Keqin Bao, Junyang Lin, and Dayiheng Liu. 2025. Chain of execution supervision promotes general reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Yangruibo Ding, Ben Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. 2024. Traced: Execution-aware pre-training for source code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Koshi Eguchi and Takuya Akiba. 2025. Making llms program interpreters via execution trace chain of thought. In *ICML 2025 Workshop on Programmatic Representations for Agent Learning (PRAL)*.

Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-guided LLM-based test generation at meta. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track*.

Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. 2024. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

Mirazul Haque, Petr Babkin, Farima Farmahinifarahani, and Manuela Veloso. 2025. Towards effectively leveraging execution traces for program repair with code LLMs. In *KnowledgeNLP Workshop at ACL 2025*.

Thong Hoang, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. DeepJIT: An end-to-end deep learning framework for just-in-time defect prediction. In *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, pages 34–45.

Thong Hoang, Hong Jin Kang, Julia Lawall, and David Lo. 2020. CC2Vec: Distributed representations of code changes. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*, pages 518–529.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of

large language models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*.

Jiangping Huang, Wenguang Ye, Weisong Sun, Jian Zhang, Mingyue Zhang, and Yang Liu. 2026. TraceCoder: A trace-driven multi-agent framework for automated debugging of LLM-generated code. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

James A. Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering (ICSE)*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering*, 39(6):757–773.

Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. In *Proceedings of the ACM on Software Engineering (FSE)*.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code execution with pre-trained language models. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4984–4999.

Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. Next: Teaching large language models to reason about code execution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Mutation testing advances: An analysis and survey. *Advances in Computers*, 112:275–378.

Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2021. JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction. In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, pages 369–379.

Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. AgentFL: Scaling llm-based fault localization to project-level context. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Rémi Lacroix, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, and 7 others. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Noveen Sachdeva, Benjamin Coleman, Wang-Cheng Kang, Jianmo Ni, Lichan Hong, Ed H. Chi, James Caverlee, Julian McAuley, and Derek Zhiyuan Cheng. 2024. How to train data-efficient LLMs. *arXiv preprint arXiv:2402.09668*.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Claudio Spiess, David Gros, and 1 others. 2025. Calibration and correctness of language models for code. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A. Smith, and Yejin Choi. 2020. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of the 2020 Conference on Empirical*

Methods in Natural Language Processing (EMNLP), pages 9275–9293.

Qwen Team. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F. Bissyandé. 2020. Evaluating representation learning of code changes for predicting patch correctness in program repair. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 981–992.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Jian Wang, Xiaofei Xie, Qiang Hu, Shangqing Liu, and Yi Li. 2025a. Do code semantics help? a comprehensive study on execution trace-based information for code large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025b. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Yunkun Wang, Yue Zhang, Guochang Li, Chen Zhi, Binhua Li, Fei Huang, Yongbin Li, and Shuiguang Deng. 2025c. InspectCoder: Dynamic analysis-enabled self repair through interactive LLM-debugger collaboration. *arXiv preprint arXiv:2510.18327*.

Ratnadira Widyasari, Jia Wei Ang, Truong Giang Nguyen, Neil Sharma, and David Lo. 2024. Demystifying faulty code: Step-by-step reasoning for explainable fault localization. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*.

W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. Software fault localization using DStar (D*). *Journal of Systems and Software*, 91:44–58.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. 2023. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ruiyang Xu and 1 others. 2025. CRUXEval-X: A benchmark for multilingual code reasoning, understanding and execution. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Aashish Yadavally, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2024. A learning-based approach to static program slicing. In *Proceedings of the ACM on Programming Languages (OOPSLA)*.

Haoran Yang, Yumeng Zhang, Jiaqi Xu, Hongyuan Lu, Pheng Ann Heng, and Wai Lam. 2024a. Unveiling the generalization power of fine-tuned large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Quanjun Zhang, Chunrong Fang, Weisong Sun, Yan Liu, Tieke He, Xiaodong Hao, and Zhenyu Chen. 2024. APPT: Boosting automated patch correctness prediction via fine-tuning pre-trained models. *IEEE Transactions on Software Engineering*, 50(3).

Li Zhong, Zilong Wang, and Jingbo Shang. 2024. LDB: A large language model debugger via verifying runtime execution step by step. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. 2024. Leveraging large language model for automatic patch correctness assessment. *IEEE Transactions on Software Engineering*, 50(11).

A Per-Class Performance Breakdown

Table 4 reports per-class precision, recall, and F1 for all experimental conditions.

B Training Details

Table 5 summarizes the training hyperparameters shared across all conditions. All experiments use the HuggingFace TRL `SFTTrainer` with LoRA (Hu et al., 2022) applied to all linear layers.

Table 6 reports per-condition training wall-clock times. Conditions with longer trace inputs (e.g., `full_trace`, VOTA) require proportionally more training time due to increased sequence lengths.

C VOTA Extraction Algorithm

Algorithm 1 describes the three-component VOTA extraction pipeline applied to a raw `sys.settrace` execution log.

Algorithm 1 VOTA Trace Extraction

Require: Raw trace T from `sys.settrace` (list of events)
Ensure: Compressed VOTA representation V

$V \leftarrow$ empty string
// Component 1: Exception summary
for each event e in T with type `exception` **do**
  Parse traceback $\rightarrow$ extract error type, message, file, line
  Append `[ERROR] type: message (file:line)` to V
end for
if no exception events found **then**
  Append `[No exception data in trace]` to V
end if
// Component 2: Assertion-point comparison
for each `assert` or `return` event e in T **do**
  Extract actual value from execution state
  Extract expected value from test assertion
  Append `[ASSERT] actual=... expected=... match=T/F` to V
end for
// Component 3: Loop compression
for each consecutive repeated line sequence S in T **do**
  if $|S| \geq 2$ iterations of same loop body **then**
    Replace S with single iteration + `[repeated ×N]`
  end if
end for
Append compressed non-exception lines to V
return V

D VOTA Trace Examples

We provide two representative VOTA trace examples illustrating the fail and pass cases. In the fail case, the exception event provides an unambiguous fault signal. In the pass case, the absence of an exception event combined with a matching assertion point confirms correct execution.

Fail example. A `ZeroDivisionError` triggers an `[ERROR]` event, providing a direct fault signal that is sufficient for the model to predict `fail`.

```
## Function
def divide(a, b):
    return a / b

## Test Input
divide(1, 0)

## Expected Output
inf

## VOTA Trace
[ERROR] ZeroDivisionError: division by zero

## Verdict
fail
```

Pass example. No exception is raised, and the assertion point confirms that the actual output matches the expected value.

```
## Function
def add(a, b):
    return a + b

## Test Input
add(2, 3)

## Expected Output
5

## VOTA Trace
[LOOP] No loops detected.
[ASSERT] actual=5, expected=5

## Verdict
pass
```

These examples illustrate the core asymmetry exploited by the `exception_only` condition: fail cases produce an explicit `[ERROR]` event, while pass cases are characterized by its absence. This binary signal (presence vs. absence of an exception) is sufficient for near-perfect verdict prediction.

E Full Experimental Results

Table 1 reports seed 42 (primary seed) results for all conditions. Table 7 extends this with multi-seed means and standard deviations where robustness seeds were run.

Seeding strategy. All conditions were run with seed 42 as the primary seed. Robustness seeds (123, 456) were added for conditions central to the paper's claims: `exception_only` and VOTA across all four models, and the full ablation set (ae_only, loop_only, label_only) on Qwen-7B. Baseline conditions (`no_trace`, `full_trace`) were run with a single seed because they serve as reference points whose absolute values matter less than the gap to Tier 1 conditions; where multi-seed baselines were run (Qwen-1.5B `no_trace`, 3 seeds), the high variance ($\pm$4.48pp) further confirms instability relative to `exception_only`'s zero variance. Conditions

Table 4: Per-class precision, recall, and F1 for all experimental conditions (seed 42). DeepSeek VOTA seed 42 is a training failure (84.45%, fail recall = 45.4%) due to tokenizer-driven truncation; seeds 123/456 achieve 99.16%/99.33%.

Condition	Model	Acc	Pass P	Pass R	Pass F1	Fail P	Fail R	Fail F1
No Trace	Qwen-2.5-7B	75.13	87.76	75.79	81.34	54.73	73.45	62.72
Full Trace	Qwen-2.5-7B	81.76	87.74	86.60	87.17	67.43	69.62	68.51
Full Trace (LO)	Qwen-2.5-7B	85.94	88.21	93.18	90.63	80.07	68.73	73.97
VOTA	Qwen-2.5-7B	99.05	100.00	99.18	99.59	97.98	100.00	98.98
Exception Only	Qwen-2.5-7B	99.83	100.00	99.76	99.88	99.41	100.00	99.71
AE Only	Qwen-2.5-7B	99.92	100.00	99.88	99.94	99.71	100.00	99.85
Loop Only	Qwen-2.5-7B	86.05	92.39	87.07	89.66	71.65	82.01	76.48
VOTA_no_ae	Qwen-2.5-7B	99.41	99.88	99.29	99.59	98.26	99.71	98.98
No Trace	DeepSeek-6.7B	73.36	81.71	80.85	81.28	53.16	54.57	53.86
Full Trace	DeepSeek-6.7B	79.16	85.70	85.10	85.40	63.20	64.30	63.70
VOTA	DeepSeek-6.7B	84.45	82.14	100.00	90.20	100.00	45.40	62.50
Exception Only	DeepSeek-6.7B	99.33	100.00	99.06	99.53	97.69	100.00	98.83
AE Only	DeepSeek-6.7B	98.74	98.40	99.90	99.10	99.70	95.90	97.70

Table 5: Training hyperparameters (shared across all conditions).

Hyperparameter	Value
LoRA rank (r)	16
LoRA alpha (α)	32
LoRA dropout	0.05
LoRA target modules	all linear layers
Epochs	3
Per-device batch size	4
Gradient accumulation steps	8
Effective batch size	32
Learning rate	2×10^{-4}
LR scheduler	linear decay
Warmup ratio	0.1
Weight decay	0.01
Max sequence length	2048 tokens
Primary seed	42
Robustness seeds	123, 456
Hardware	1× RTX PRO 6000 Blackwell (96 GB VRAM)
Framework	HuggingFace TRL SFTTrainer

Table 6: Training wall-clock time per condition on a single RTX PRO 6000 Blackwell GPU. Where multiple seeds were run, individual times are listed.

Condition	Seed(s)	Time (h)
no_trace	42	∼0.5
full_trace	42	∼2.2
label_only	42	∼2.2
VOTA	42	∼2.3
exception_only	42	∼2.1
exception_only	123	∼2.1
exception_only	456	∼1.3
ae_only	42	∼1.8
loop_only	42	∼2.1
VOTA_no_ae	42	∼1.8

specific to a single analysis (`VOTA_no_ae`, gradient masking) were run with a single seed to conserve compute.

Tables 8 and 9 report the complete per-seed results for Qwen-7B and DeepSeek-6.7B.

Table 7: Cross-model multi-seed results (%, N=1,190). Mean±std across available seeds; entries without ± are single-seed. ‡2-seed mean (seed 42 training failure excluded; §F).

Condition	Qwen-7B	Qwen-1.5B	CL-13B	DS-6.7B
exception_only	99.83±0.00	99.83±0.00	99.89±0.10	99.39±0.10
ae_only	99.64±0.49	98.83±0.36	99.08±0.00	98.53±0.30
VOTA	99.05±0.38	99.54±0.06	98.74±0.24	99.25‡
loop_only	86.05±0.52	79.54±1.42	77.40±1.54	80.31±1.65
no_trace	75.13	72.35±4.48	76.47	73.36
Majority baseline		71.51		

Table 8: Full results for Qwen-2.5-Coder-7B across all conditions and seeds. Conditions with a single seed report no variance. `exception_only` achieves 99.83% across all three seeds (σ=0). `ae_only` achieves 99.64% mean (±0.49; seed 123 at 99.08% is the outlier).

Condition	S42	S123	S456	Mean ± Std
exception_only	99.83	99.83	99.83	99.83 ± 0.00
ae_only	99.92	99.08	99.92	99.64 ± 0.49
VOTA	99.41	98.66	99.08	99.05 ± 0.38
VOTA_no_ae	99.41	—	—	—
label_only	86.22	86.89	84.70	85.94 ± 1.12
loop_only	85.63	85.88	86.64	86.05 ± 0.52
full_trace	81.76	—	—	—
no_trace	75.13	—	—	—

Table 9: Full results for DeepSeek-Coder-6.7B across all conditions and seeds. `exception_only` is consistent across seeds: 99.33/99.50/99.33 (σ=0.10pp).

Condition	Seed	Accuracy	Notes
exception_only	42	99.33	epoch 1: 99.83
exception_only	456	99.33	CI [98.82, 99.75]
exception_only	123	99.50	
VOTA	42	84.45	training failure; fail recall = 45.4%
VOTA	123	99.16	
VOTA	456	99.33	
no_trace	42	73.36	
full_trace	42	79.16	CI [76.80, 81.30]

F Tokenizer and Truncation Analysis

DeepSeek VOTA seed 42 (84.45%) is a training failure driven by tokenizer-induced truncation. Re-evaluation of the seed 42 checkpoint with the standard max_length=2048 yields 16% accuracy, which confirms complete model collapse rather than marginal degradation. The previously reported 84.45% was obtained with max_length=16384, which partially bypasses tokenizer truncation. Seeds 123 and 456, evaluated under identical conditions, achieve 99.16% and 99.33% respectively. The DeepSeek tokenizer averages 2,626 tokens per VOTA sample versus 1,929 for Qwen, causing 91.7% of DeepSeek VOTA training samples to exceed the 2,048-token context window. The training framework preserves the verdict label but truncates trace content, so the model trains with incomplete diagnostic information for the majority of samples. Seeds 123 and 456 (99.16% and 99.33%) demonstrate that DeepSeek can learn VOTA when LoRA initialization avoids the truncation-sensitive basin; the 2-seed mean (99.25%$\pm$0.12) matches the cross-model VOTA range (98.74–99.54%).

A post-hoc group analysis on seed 42 provides directional evidence: partitioning test samples by whether their training-set counterparts exceeded the context window, intact samples (N=99) achieve 100% accuracy while truncated samples (N=1,091) achieve 83.04% (95% CI [80.75%, 85.24%]; Fisher exact $p<.001$), with all 185 errors concentrated in the fail class (fail recall = 31.23%). Per-class analysis (Table 4) confirms that seed 42 achieves 100% fail precision but only 45.43% fail recall, systematically defaulting to `pass`.

Seed robustness is consistent across models: Qwen `exception_only` produces identical 99.83% across all three seeds (σ=0), and DeepSeek achieves 99.39%$\pm$0.10 (seeds 42/456: 99.33%, seed 123: 99.50%). Because `exception_only` test inputs contain 48.99% duplicates (607 unique of 1,190; §3.2), CIs assume n=1,190 independent trials; under deduplication (n=607), Qwen's accuracy is 99.67% (Wilson CI [98.81, 99.91]).

G Irreducible Sample Analysis

The *exception_only* condition with Qwen-2.5-Coder-7B misclassifies exactly 2 out of 1,190 test samples across all three seeds (42, 123, 456), with 100% overlap across seeds. Both samples (IDs 901 and 905) share the following characteristics:

- **Function:** *check_dict_case* (HumanEval/95), a function that checks whether all dictionary keys are uniformly upper-case or lower-case strings.
- **Ground-truth label:** *pass*.
- **Predicted label:** *fail* (false negative).
- **Cross-seed consistency:** Both samples are misclassified identically by all three seeds, which indicates a systematic rather than stochastic error.

Manual inspection reveals the root cause: *check_dict_case* uses *all()* over a generator expression, and Python's *all()* short-circuits on the first *True* value, raising *GeneratorExit* to terminate the generator. Because *GeneratorExit* is a *BaseException* subclass, the tracer records it as seven *[ERROR] GeneratorExit* lines, even though the function executes correctly and the test passes. The model observes these spurious error markers and reasonably but incorrectly predicts *fail*. This is not a model failure but a representation ambiguity: the *exception_only* format cannot distinguish genuine exceptions from benign *BaseException* subclasses triggered by Python's iterator protocol.

Most configurations converge to these same 2 errors: Qwen-1.5B, Qwen-7B (3 seeds), CodeLlama-13B (seeds 42/456), and DeepSeek-6.7B (seeds 42/456) all produce error set $\{901, 905\}$. However, CodeLlama-13B seed 123 achieves 100% accuracy (0/1190 errors), correctly classifying both samples. This demonstrates that 99.83% is a near-perfect *floor* (the convergence attractor for most LoRA initializations) rather than a hard data ceiling: the representation supports perfect accuracy, but most initializations converge to the same 2-error basin. The practical implication is that these errors are high-probability convergence artifacts addressable through seed selection or ensemble methods, not fundamental limitations of the exception-based representation.

H Exception Type Accuracy Breakdown

Table 10 reports `exception_only` accuracy stratified by exception type on Qwen-2.5-7B (seed=42, N=1,190). All standard exception types (`TypeError`, `ValueError`, `IndexError`, etc.) achieve 100% accuracy. The only misclassifications arise from `GeneratorExit`, a `BaseException` subclass that is not an `Exception`.

It is raised by Python's iterator protocol during generator short-circuiting, not by actual program errors.

Table 10: Exception type accuracy breakdown for `exception_only` (Qwen-7B, seed=42). All true error types achieve 100%. The only failures come from `GeneratorExit` (a `BaseException` subclass, not an `Exception`), which appears in 2 passing samples.

Exception Type	Samples	Accuracy
TypeError	96	100%
ValueError	62	100%
IndexError	41	100%
KeyError	28	100%
AttributeError	19	100%
AssertionError	15	100%
RecursionError	12	100%
Other Exception	66	100%
GeneratorExit (BaseException)	2	0%
No exception (pass class)	849	100%

This confirms that the model has learned a clean decision boundary: any true `Exception` subclass in the trace implies failure. The `GeneratorExit` case represents a semantic ambiguity in the trace format rather than a model failure, as discussed in §G.

I Trace Length Ablation

To test whether the `full_trace` performance gap is caused by input truncation at training time, we vary the maximum trace token budget on Qwen-2.5-7B (seed=42) while keeping all other hyperparameters fixed.

Table 11: Trace length and truncation direction ablation on Qwen-2.5-7B (N=1,190, seed=42). Head = standard left-truncation (retain first k tokens); Tail = retain last k tokens. Per-condition bootstrap 95% CIs (B=10,000).

Direction	Max Tokens	Acc (%)	95% CI
Head	256	78.82	[76.4, 81.0]
Head	512	80.42	[78.2, 82.6]
Head	1024	77.23	[74.9, 79.6]
Head	no_limit (∞)	81.76	[79.7, 83.9]
Tail	256	77.48	—
Tail	512	93.61	—
Tail	1024	96.47	—
Tail	2048	97.31	—

As shown in Table 11 and Figure 3, all four trace length conditions fall within a 5pp range (77.2%–81.8%). To quantify the reliability of the observed differences, Table 12 reports paired bootstrap comparisons (B=10,000, seed=42) across all condition pairs.

Table 12: Pairwise paired bootstrap comparisons for trace length ablation (B=10,000, seed=42). Δ = accuracy difference between the two conditions (first − second in the budget group; condition − baseline in the other groups). $^{**}p<.01$; $^{***}p<.001$.

	Comparison	Δ (pp)	95% CI	p-value
Budget	512 vs. 256	+1.60	[−0.34, +3.53]	0.117
	1024 vs. 512	−3.19	[−5.29, −1.09]	0.002**
	no_limit vs. 1024	+4.54	[+2.52, +6.55]	<0.001***
	no_limit vs. 256	+2.94	[+0.84, +5.04]	0.006**
vs. no_trace	256 − no_trace	+3.70	[+1.43, +6.05]	0.002**
	512 − no_trace	+5.29	[+3.03, +7.56]	<0.001***
	1024 − no_trace	+2.10	[−0.17, +4.37]	0.078
	no_limit − no_trace	+6.64	[+4.37, +8.99]	<0.001***
vs. exc.	256 − exc_only	−21.01	[−23.45, −18.74]	<0.001***
	512 − exc_only	−19.41	[−21.76, −17.14]	<0.001***
	1024 − exc_only	−22.61	[−25.04, −20.25]	<0.001***
	no_limit − exc_only	−18.07	[−20.34, −15.88]	<0.001***

Non-monotonic pattern (head-truncation). Under head-truncation (retaining trace prefixes), the trend is non-monotonic: 512 tokens (80.42%) slightly outperforms 256 tokens (78.82%), but doubling to 1024 tokens *decreases* accuracy to 77.23%, a statistically significant 3.19pp drop (p=.002, 95% CI [−5.29, −1.09]). The no-limit condition (81.76%) recovers and significantly exceeds 1024 tokens ($p<.001$), but 1024 tokens is not significantly better than `no_trace` (75.13%; Δ=+2.10pp, p=.078, 95% CI [−0.17, +4.37]). This non-monotonicity is specific to head-truncation: the 1024-token prefix retains enough irrelevant content to mislead the model while discarding diagnostic events that appear near the trace terminus. Under tail-truncation, accuracy increases monotonically for Qwen-7B and Qwen-1.5B (Table 13); only DeepSeek-6.7B shows a mild −1.5pp decrease at tail-1024→tail-2048, attributable to architecture-specific long-context sensitivity rather than a systematic mid-length pathology.

Comparison to baselines. All four trace budget conditions significantly exceed `no_trace` except 1024 tokens (p=.078), which confirms that raw trace content provides some signal but unreliably so. The gap to `exception_only` (99.83%) is massive for every budget condition (18–23pp, all $p<.001$), underscoring that even unlimited raw traces cannot substitute for targeted signal extraction. The best full-trace condition (no_limit, 81.76%) still lags `exception_only` by 18.07pp (95% CI [−20.34, −15.88]).

Interpretation. These results rule out input truncation as the primary explanation for the full-trace performance gap: even with unlimited tokens, the model cannot extract the diagnostic signal that VOTA's targeted compression provides. The non-monotonic dip at 1024 tokens is consistent with a "worst-of-both-worlds" regime where the model receives enough noisy context to dilute the signal but not enough to compensate through broader coverage. The result reinforces the information density interpretation (§4.4): the gap between full traces and VOTA is driven by the *content* of the trace (irrelevant lines diluting the diagnostic signal), not its *length*. In contrast, VOTA's compressed representation concentrates the diagnostic signal into fewer tokens, making it sensitive to truncation when those tokens are removed (§4.1).

Truncation direction. Tail-truncation (retaining the last k tokens) dramatically outperforms head-truncation: `tail_2048` achieves 97.31% vs. `head_2048` (no_limit) at 81.76%, a 15.55pp improvement. Even `tail_512` (93.61%) exceeds every head-truncation condition by over 11pp.

However, `tail_256` (77.48%) performs at the `no_trace` baseline level (75.13%). This indicates that the diagnostic signal does not reside in the very last trace lines. The critical jump occurs between 256 and 512 tail tokens (+16.13pp), localizing the signal window to approximately 256–512 tokens from the trace terminus. This is consistent with Python's execution model: exception tracebacks and assertion comparisons appear near the end of the trace but are preceded by the final function call frames, placing them typically 300–500 tokens from the terminus rather than at the absolute end.

The 18.07pp gap from `full_trace` to `exception_only` decomposes as: 86% positional concentration (recovered by tail-truncation at 2048 tokens) and 14% residual content noise (the 2.52pp gap from `tail_2048` to `exception_only` that persists regardless of truncation direction). The fact that `exception_only` (99.83%) still outperforms `tail_2048` (97.31%) confirms that semantic extraction (identifying the exception type and message) is superior to positional extraction (keeping a fixed window), because the model need not filter out non-diagnostic lines that happen to fall within the window.

Cross-model budget curve. Table 13 extends the tail-truncation analysis to a full budget sweep across four models spanning three architecture families and an 8.7× parameter range.

Table 13: Cross-model tail-truncation budget curve (N=1,190). All four models exhibit a transition between tail-384 and tail-512 (+13–23pp). ↓: non-monotonic decrease. †CodeLlama tail-1024 trained with label-only loss to address tokenizer-driven gradient dilution (§F). ‡CodeLlama tail-2048 outlier (41.8%): LoRA long-sequence instability under label-only loss; excluded from cross-model analysis.

Model	tail-256	tail-384	tail-512	tail-1024	tail-2048	exc_only
Qwen-7B (7B)	78.9	79.9	93.6	96.5	97.3	99.83
DeepSeek (6.7B)	76.4	82.9	93.9	96.5	95.0↓	99.33
CodeLlama (13B)	77.9	80.6	98.2	98.2†	41.8‡	99.89
Qwen-1.5B (1.5B)	72.6	73.6	95.6	96.6	97.9	99.83

Table 14: Cross-model truncation direction comparison (N=1,190). Head-truncation retains the first 2,048 tokens; tail-truncation retains the last 2,048. All four models' head-2048 accuracies converge into a narrow 75.3–81.8% band (6.5pp), comparable to the tail-256 signal-poor band (72.6–78.9%, 6.3pp). †CodeLlama tail-2048 pending retraining (§F).

Model	head-2048 (%)	tail-2048 (%)	Gap (pp)
Qwen-1.5B	75.3	97.9	+22.6
DeepSeek-6.7B	78.7	95.0	+16.3
CodeLlama-13B	79.9	—†	—†
Qwen-7B	81.8	97.3	+15.5

Four patterns emerge from the budget curve. *Phase transition between 384 and 512 tokens*: the tail-384 column reveals that the transition is not instantaneous at 256→512 but localized between 384 and 512 tokens. At tail-384, all four models remain near baseline (73.6–82.9%), gaining only +1.0–6.5pp over tail-256. The steep jump occurs at 384→512: +13.7pp (Qwen-7B), +11.0pp (DeepSeek), +17.6pp (CodeLlama), +22.0pp (Qwen-1.5B), crossing from near-baseline to near-ceiling. This localizes the critical threshold to ∼400–500 tokens, consistent with the typical exception traceback length: below ∼384 tokens, the traceback is truncated and no diagnostic event is captured; above ∼512, it is fully included regardless of model capacity or architecture.

Convergence at tail-512+: once the phase transition is crossed, accuracy converges across all models into a 93.6–98.2% band at tail-512 (seed 42). Multi-seed validation confirms robustness: across all 12 model×seed combinations at tail-512 (Table 15), all accuracies fall within [93.61%, 98.24%], a 4.63pp band that compresses the signal-poor tail-256 spread (6.3pp) by 27%. CodeLlama-13B shows perfect per-model convergence at tail-

512: all three seeds achieve 98.24% (σ=0), consistent with the 13B scale and label-only loss providing a stronger optimization signal in the signal-rich regime (Table 15).

Table 15: Cross-seed convergence at the tail-512 truncation point (N=1,190 per seed). All 12 model×seed combinations fall within a 4.63pp band [93.61%, 98.24%]. CodeLlama-13B achieves identical accuracy across all three seeds (σ=0).

Model	S42	S123	S456	Mean	σ
Qwen-7B (7B)	93.61	96.00	97.65	95.75	2.05
DeepSeek (6.7B)	93.95	95.04	96.10	95.03	1.08
Qwen-1.5B (1.5B)	95.63	97.65	96.72	96.67	1.01
CodeLlama (13B)	98.24	98.24	98.24	**98.24**	**0.00**
Cross-model band				[93.61, 98.24] = 4.63pp	

At tail-1024, four models reach 96.5–98.2% (CodeLlama 98.2% under label-only loss, three others 96.5–96.6%). Scale and architecture differences that dominate the signal-poor regime (tail-256 range: 72.6–78.9%, 6.3pp) compress dramatically in the signal-rich regime.

Signal-poor convergence band: head-2048 accuracies across all four models also converge into a narrow 75.3–81.8% band (6.5pp; Table 14), comparable in width to the tail-256 band (72.6–78.9%, 6.3pp). When the diagnostic signal is absent (head-truncation discards trace terminus) or unreachable (tail-256 is too short), models of different scale and architecture collapse to similar performance, suggesting a shared floor determined by code comprehension rather than trace processing.

Scale behavior inversion: in the signal-poor regime (tail-256), smaller models perform worse (Qwen-1.5B 72.6% vs. Qwen-7B 78.9%, −6.3pp), mirroring the `no_trace` scale pattern. In the signal-rich regime (tail-512+), this reverses: Qwen-1.5B matches or exceeds Qwen-7B (95.6% vs. 93.6% at tail-512; 97.9% vs. 97.3% at tail-2048), which confirms that sufficient trace signal eliminates the capacity advantage.

Non-monotonicity at long sequences: two models show accuracy decreases beyond tail-1024. DeepSeek-6.7B drops −1.5pp from tail-1024 (96.5%) to tail-2048 (95.0%), consistent with its known long-context sensitivity (§N). CodeLlama-13B exhibits a more severe collapse: tail-2048 under label-only loss drops to 41.8% (vs. tail-1024 = 98.2%), which we attribute to LoRA long-sequence instability under the label-only loss regime; this outlier is excluded from cross-model tail-2048 analysis. Qwen-7B and Qwen-1.5B increase monotonically across all budgets.

Table 14 confirms the truncation direction effect: among the three models with complete data, tail-2048 outperforms head-2048 by 15.5–22.6pp. The gap is largest for Qwen-1.5B (+22.6pp). This indicates that smaller models benefit most from positional proximity to the diagnostic signal, as they lack the capacity to locate it within longer, noisier prefixes.

J Calibration Analysis

Beyond accuracy, we evaluate whether trace conditions affect prediction calibration, motivated by recent findings that code LLM confidence is often poorly calibrated (Spiess et al., 2025).

Table 16: Calibration metrics on Qwen-2.5-7B (N=1,190, seed=42). ECE = expected calibration error; Brier = Brier score. Lower is better for both.

Condition	ECE ↓	Brier ↓
ae_only	**0.0004**	**0.0008**
exception_only	0.0027	0.0013
VOTA	0.0054	0.005
full_trace	0.096	0.144
no_trace	0.171	0.197

Table 16 shows that calibration quality mirrors the Tier 1/2 hierarchy. Tier 1 conditions (`ae_only`: ECE=0.0004; `exception_only`: ECE=0.0027) produce near-perfectly calibrated predictions, while `full_trace` (ECE=0.096) and `no_trace` (ECE=0.171) exhibit substantial miscalibration. VOTA (ECE=0.0054) is well-calibrated but slightly worse than either Tier 1 component alone, reinforcing the pattern that combining trace signals dilutes rather than improves the prediction signal.

The Brier scores tell a consistent story: `ae_only` (0.0008) and `exception_only` (0.0013) achieve near-zero scores, while `no_trace` (0.197) indicates substantial miscalibration.

Cross-model calibration. Table 17 extends the calibration analysis to four models (1.5B–13B) under `exception_only`. All four models achieve ECE < 0.007 and Brier < 0.007, which confirms that calibration quality generalizes across architectures and scales. CodeLlama-13B achieves the lowest ECE (0.0020) and MCE (0.0411), while DeepSeek-6.7B shows the highest ECE (0.0064) and Brier (0.0061), consistent with its higher error rate (8 errors vs. 2 for Qwen/CodeLlama).

Table 17: Cross-model calibration for `exception_only` (N=1,190, seed=42). ECE = expected calibration error; Brier = Brier score; MCE = maximum calibration error. All four models (1.5B–13B) achieve ECE < 0.007.

Model	ECE ↓	Brier ↓	MCE ↓
Qwen-1.5B	**0.0020**	**0.0013**	—
CodeLlama-13B	**0.0020**	**0.0013**	**0.0411**
Qwen-7B	0.0027	**0.0013**	0.8355
DeepSeek-6.7B	0.0064	0.0061	0.7756

K Error Overlap Analysis

We analyze the overlap between error sets of different conditions to determine whether they fail on the same samples (Figure 5).

`exception_only`'s 2 errors (samples 901 and 905) have zero intersection with the error sets of all other conditions (Jaccard = 0.000 in all pairwise comparisons). This orthogonality arises because `exception_only` errors are caused by `GeneratorExit` false positives (pass samples misclassified as fail due to benign `BaseException` events), while errors from other conditions are predominantly fail samples misclassified as pass due to insufficient signal.

Among non-exception conditions, error sets exhibit hierarchical containment: `ae_only` $\subset$ VOTA $\subset$ `loop_only` $\subset$ `no_trace`, with pairwise Jaccard indices increasing with signal overlap. `loop_only` and `full_trace` share the highest overlap (Jaccard $\sim$0.65), reflecting their shared reliance on behavioral patterns rather than explicit fault indicators.

This analysis confirms that `exception_only` operates on an independent signal dimension: its errors are representation ambiguities (iterator protocol artifacts) rather than information deficits, explaining why combining it with other signals cannot eliminate these specific errors.

L Cross-Dataset Generalization: DebugBench

To test whether exception-based verdict prediction generalizes beyond LLM-generated solutions, we evaluate on DebugBench (Tian et al., 2024), a dataset of human-written programs with synthetically injected bugs (single-point mutations such as operator replacement, variable misuse, and missing statements). After filtering for executable Python samples with valid traces, we retain 426 samples with a near-balanced class distribution (majority baseline: 51.4%).

Table 18: Generalization to DebugBench (N=426, Qwen-2.5-7B, seed=42). Wilson 95% CIs. The component ranking (exception_only $\geq$ VOTA $>$ no_trace) is preserved, and exception_only achieves perfect accuracy.

Condition	Acc (%)	95% CI
exception_only	**100.00**	[99.14, 100.00]
VOTA	99.77	[99.30, 100.00]
no_trace	96.71	[94.80, 98.40]
Majority baseline	51.41	—

Table 18 shows that the component ranking from the main experiments is preserved: `exception_only` (100.00%) $\geq$ VOTA (99.77%) $>$ `no_trace` (96.71%). The `exception_only` condition achieves perfect accuracy. This confirms that the exception-based representation generalizes to a different fault distribution without degradation.

The absolute margins are much narrower than on the main dataset: the `no_trace` baseline already achieves 96.71% (compared to 75.13% on HumanEval+MBPP), leaving only 3.29pp of headroom for trace-based improvement. This ceiling effect reflects a fundamental difference in fault characteristics: DebugBench bugs are synthetically injected single-point mutations, which produce code that is structurally dissimilar from the original correct solution; a model can often identify the fault from source code alone. In contrast, the HumanEval+MBPP solutions are LLM-generated, producing natural errors (logic mistakes, edge-case failures) where the faulty and correct solutions may be structurally similar, making code-only verdict prediction substantially harder.

These results support exception-based verdict prediction as a generalizable approach: the method does not degrade on an out-of-distribution dataset, and the exception dominance ranking (exception_only $\geq$ VOTA $>$ no_trace) is consistent across both datasets. However, the narrow DebugBench margins are insufficient to independently validate trace importance. The 24.70pp trace lift on the main dataset (no_trace 75.13% $\rightarrow$ exception_only 99.83%) remains the primary evidence for the value of exception-based trace signals.

M Cross-Model Robustness Analysis

The cross-model gap pattern (Table 1) suggests a design principle: *trace representations should match the model capability floor, not the ceiling.* Exception events are consistently the most

effective component on both models, and their structural simplicity (a fixed-format error message) ensures that the relative advantage over richer representations holds across all seeds. All three DeepSeek `exception_only` seeds achieve $\geq$99.33% (99.33/99.50/99.33, σ=0.10pp), matching Qwen and CodeLlama at $\geq$99.83%. VOTA achieves $\geq$98.57% across all models when excluding outlier seeds (cross-model range: 0.80pp), but DeepSeek VOTA exhibits seed-dependent instability: seed 42 (84.45%) is a training failure due to tokenizer-induced truncation (91.7% of training samples affected), while seeds 123/456 (99.16%/99.33%) match cross-model performance. This VOTA instability is confounded by tokenizer efficiency and base model differences (Yang et al., 2024a). Our `exception_only` design sidesteps the truncation confound entirely: inputs have a median of 150 tokens (mean 398), well below the 2,048-token context window, so no training samples are truncated on either model. The resulting 0.50pp cross-model gap (Table 1) is therefore a clean measure of model capability differences, making the "capability floor" principle directly testable. The practical implication is that trace-based systems should default to the simplest sufficient signal.

N Training Dynamics

The DeepSeek `exception_only` results are consistent across all three seeds (99.33/99.50/99.33, σ=0.10pp), matching Qwen's cross-seed stability. Epoch-level analysis shows mild degradation in seed 42: accuracy peaks at 99.83% after epoch 1 but settles at 99.33% by epoch 3 ($-$0.50pp), while seed 456 shows no degradation (99.33% at both epochs). Early stopping may yield marginal gains, but the epoch effect is small ($\leq$0.5pp). The practical recommendation: for deployment, use task-specific metrics (accuracy, fail recall) for checkpoint and seed selection rather than training loss, and train multiple seeds to select the best.

The contrast with VOTA is instructive: DeepSeek VOTA seed 42 accuracy monotonically increases from 78.15% to 84.45% across epochs, consistent with a performance limit from high trace truncation (91.7% of samples; §F). Seeds 123/456, which avoid the truncation-sensitive initialization basin, achieve 99.16%/99.33%. This confirms that the deficit is seed-specific rather than a fundamental model limitation.

O Deduplication Statistics

Table 19 reports the duplication rates across trace formats. Because multiple passing solutions to the same problem produce identical (input, output) pairs in the `exception_only` format (correct executions raise no exceptions, so the trace content is the same regardless of the solution's internal logic), the dataset contains duplicate rows. Train–test input overlap is zero (split by problem ID), so no information leakage occurs. The high duplication in `exception_only` does not affect our experimental conclusions: after deduplication the training set still contains 2,858 unique samples and the test set 607, providing sufficient coverage for the 591-problem benchmark.

Problem-level partitioning and sensitivity analysis. Data is partitioned at the problem level: 472 training problems and 119 test problems, with zero crossover. Duplicates arise from multiple candidate solutions per problem (up to 10 solutions each), not from cross-problem overlap. To verify that deduplication does not alter any finding, we ran a sensitivity analysis across all 19 experimental conditions (7 trace formats $\times$ 4 models, minus conditions not run): deduplication produces a 0.00pp accuracy delta in every case. This confirms that duplicate samples neither inflate nor deflate accuracy, because duplicates share the same ground-truth label and the model's prediction is deterministic at inference time. The `exception_only` test set contains 48.99% duplicate inputs due to representation compactness; all confidence intervals treat samples as independent Bernoulli trials.

Table 19: Deduplication statistics. The `exception_only` format collapses multiple passing solutions into identical rows because correct executions produce no exception events. Train–test input overlap is 0 (no leakage).

Split	Total	Unique	Dup. rate
exception_only train	4,720	2,858	39.45%
exception_only test	1,190	607	48.99%
raw_traces (full)	5,910	5,803	1.81%

P Regex Baseline Analysis

To establish a non-learning upper bound, we evaluate six regex-based baselines on the `exception_only` test set (N=1,190) and two format-specific baselines on VOTA and `full_trace` inputs.

Table 20: Regex baseline accuracy (N=1,190). B4 is the strongest semantic baseline (6.22pp below SFT `exception_only`, McNemar $p<.001$). B6 uses an error count threshold. ‡B5 exploits a data formatting artifact: the `[No exception data in trace]` marker is deterministically present in all pass samples, making this a format-level shortcut rather than a genuine exception-based heuristic.

Baseline	Method	Acc (%)
B1: `[ERROR]` presence	string match	93.45
B2: Traceback presence	string match	71.51
B3: Any keyword	union of B1+B2+Exception	93.45
B4: Exception type regex	regex on exception class names	**93.61**
B5: Format marker	`[No exception data]` → pass	99.33‡
B6: Error count ≥2	count threshold	92.02
VOTA format regex	best regex on VOTA traces	75.55
full_trace format regex	best regex on full traces	77.39

The strongest legitimate baseline (B4, 93.61%) applies regex pattern-matching on Python exception class names (e.g., `ValueError`, `TypeError`). Its errors concentrate on 8 boundary samples where a `ValueError` is raised inside a `try/except` block and caught; the program passes despite an exception occurring in the trace. SFT `exception_only` correctly classifies 6 of these 8 samples. This indicates that the model learns the semantic distinction between caught (benign) and uncaught (fault-indicating) exceptions. B5 (99.33%) achieves near-SFT accuracy but relies on the deterministic presence of a format marker in pass-class traces, which is an artifact of the trace preprocessing pipeline rather than a generalizable signal; the McNemar test vs. SFT (p=.041) confirms a statistically significant difference.

Cross-format comparison. Table 21 compares SFT and the best applicable regex baseline for each trace format. The SFT advantage increases with format complexity: `no_trace` (+3.62pp over majority baseline) < `full_trace` (+4.37pp) < `exception_only` (+6.22pp) ≪ VOTA (+23.86pp). The VOTA gap is largest because its structured multi-component format carries semantic content that regex cannot parse, while `exception_only`'s simpler format allows regex to capture most of the signal.

Q Cross-Model Zero-Shot Comparison

Table 22 compares zero-shot and SFT `exception_only` accuracy across four models. Zero-shot performance does not correlate with model size: the smallest model (Qwen-1.5B, 93.45%) outperforms all larger models, while the largest (CodeLlama-13B) falls below majority baseline. SFT compresses the 42.44pp zero-shot range to 0.50pp, eliminating architecture-dependent variance.

Table 21: SFT vs. best regex baseline by trace format (N=1,190). Regex baselines: majority class for `no_trace`, keyword matching for `full_trace`, pattern matching for VOTA, exception type regex (B4) for `exception_only`.

Format	SFT (%)	Regex (%)	Δ (pp)
exception_only	99.83	93.61	+6.22
VOTA	99.05	75.55	+23.50
full_trace	81.76	77.39	+4.37
no_trace	75.13	71.51	+3.62

Table 22: Zero-shot vs. SFT `exception_only` across four models (N=1,190). Zero-shot accuracy is uncorrelated with model size; SFT normalizes cross-architecture variance from 42.44pp to 0.50pp.

Model	Size	Zero-shot (%)	SFT (%)	Δ (pp)
Qwen-1.5B	1.5B	93.45	99.83	+6.38
DeepSeek-6.7B	6.7B	90.59	99.39	+8.80
Qwen-7B	7B	89.08	99.83	+10.75
CodeLlama-13B	13B	51.01	99.89	+48.88
Range	—	42.44pp	0.50pp	—